Freight Rate Prediction

Approach, validation and the fixed December chart | Machine Learning Engineer assessment

1. Summary

The task is to price 12,000 loads for November and December 2025 from 48,000 labelled loads from January to October. The validation loads therefore lie entirely in the future of the training data, and 8 of the 72 cities in them never appear in training. I built the validation around those two facts instead of using a random split.

- **Model.** log(rate per mile) = a gradient-boosted tree on distance, equipment, weight and geography + a small linear part for the calendar and market index + shrunk offsets for known cities and lanes. The calendar part is linear so it extrapolates into months the model has not seen.
- **Accuracy, out of time.** Over six expanding-window folds (train on everything earlier, predict the next two months) the typical error is 1.5% MAPE / $36 MAE on correctly labelled loads, against 5.4% / $120 for boosted trees on the raw columns and 8.3% / $165 for a median rate-per-mile rule.
- **Label noise.** 677 training rates (1.41%) are corrupted: off by a factor of 2.3-5.3 or 0.18-0.44, with no link to any feature. They are removed from training. Any metric computed on all rows, including corrupted ones, is limited to roughly 3.8% MAPE for this reason, whatever the model.
- **Two columns could not be taken at face value.** quote_signal relates to the rate with a sign that flips from month to month (mostly because it tracks the time level) and looks inactive in the validation months, so it is excluded; market_index is replaced by its daily average because its per-load noise carries no information.
- **December.** The fixed Lexington to Fort Wayne chart rises from $835 to $885 through the month: a quarter-end ramp, a slow drift and a weekly cycle from the market index (section 6).

2. What the data showed

- **The rate is almost a clean product of a few effects.** After the model's terms, the remaining noise on correctly labelled loads is about 1.5% (standard deviation of the log error), and its tails are Gaussian. Rate per mile falls with distance, Reefer and Flatbed carry premiums of about 12% and 8% over Dry Van, the rate per mile rises with weight (about 9% from the lightest to the heaviest loads, flattening above 40,000 lb), and the market index has an elasticity of about 0.15.
- **Geography is a smooth, mostly additive effect.** A city adds roughly the same premium as a pickup and as a delivery (correlation 0.99 between the two), running from about -3% in the Great Lakes and Northeast to +3% in Texas and Oklahoma. Predicting a held-out city's premium from its three nearest cities' coordinates recovers about 88% of its variance, which is what lets the model price unseen cities.
- **The calendar matters, in two ways.** There is a slow upward drift (about 2.0% per 100 days) and a ramp in the last month of each quarter. During March, June and September the rate climbs by 2.3% (Dry Van), 4.9% (Reefer) and 7.7% (Flatbed) between the 1st and the last day, then resets at the start of the next quarter (roughly 1 : 2 : 3 across the three equipment types). December is the next quarter-end month. I found no holiday effect: the days around Memorial Day, July 4 and Labor Day move the daily level by less than 1%, in line with ordinary day-to-day noise, and load volumes do not dip either.

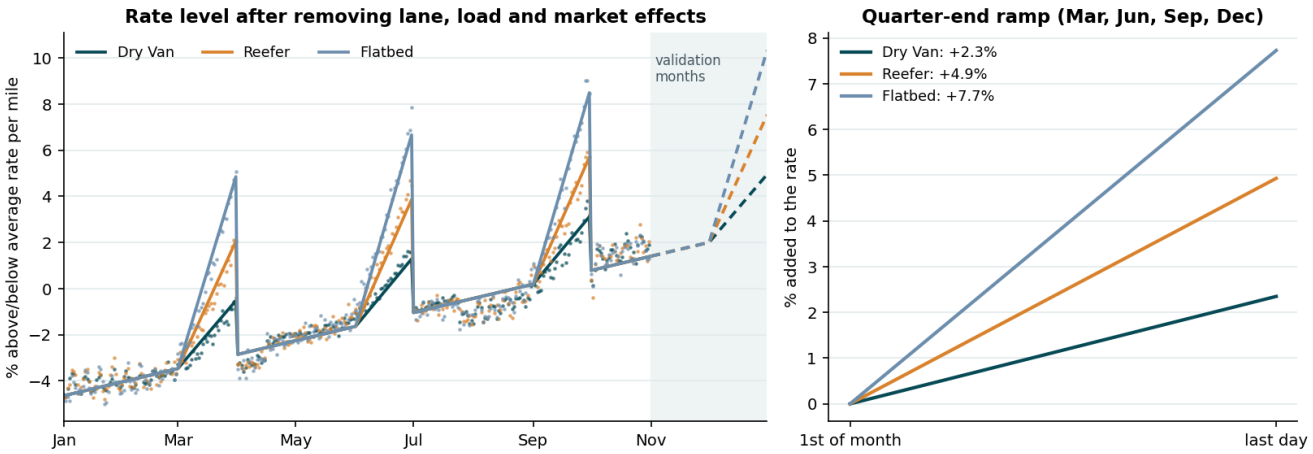


Figure 1. Left: daily average of the rate after removing the model's lane, load and market effects, with the fitted calendar term (dashed after October). Right: size of the quarter-end ramp.

3. Data-quality issues and how I handled them

Issue	Evidence (development / validation)	Handling
Corrupted rates	677 rows (1.41%); 340 are x2.3-5.3 too high and 337 are x0.18-0.44 too low. Spread evenly over months, equipment and distance (Fig. 2).	Detected from the fit (error beyond 0.25 in log terms; the largest error on any other row is 0.08, the smallest on a corrupted row 0.81) and left out of training. Reported separately: metrics on clean rows and on all rows.
quote_signal is unstable	Within each month it correlates with the rate error, but the sign changes: + in Jan, Feb, Mar, Jun, Sep; - in Apr, May, Jul, Oct; about 0 in Aug. Against a simple baseline (distance, equipment, market index) |r| is 0.26-0.41; once the calendar level is modelled it falls to 0.07-0.14 with the same signs, because it mostly tracks the time level. Its shape follows the sign (right-skewed, left-skewed, symmetric), and Nov and Dec are symmetric like Aug (Fig. 3).	Not used. Adding it to the final model raises out-of-time error from 0.019 to 0.061 (worst fold: 0.24).
market_index noise and gaps	Per-load value = daily level + noise (sd 0.025), and that noise has no relation to the rate (correlation 0.003). Missing: 374 / 249.	Replaced by the mean of that day's loads. Also provides the market level for the December chart dates.
Negative weights	292 / 145 loads. Their magnitudes match the positive ones.	Absolute value. The model's error on these rows equals its error elsewhere.
Missing weight	300 / 165 loads; no bias in the model's error on them.	Filled with the training median.
Clipped values	Weight capped at 47,500 (1204 loads, 2.5%); distance floored at 70 miles (48 / 21 loads).	Kept as given; the validation set is clipped the same way.
Cities missing from training	8 cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego), 1,447 validation loads (12%).	Priced from coordinates; lane offsets give them no correction. Tested explicitly (section 4).
Synthetic coordinates	Positions are internally consistent per city but not real (e.g. Los Angeles at 28.6 N), and distance cannot be recomputed from them.	Used only as relative position; distance taken as given.
December chart rows lack inputs	The chart file has city names, no coordinates and no market index.	Coordinates looked up from the loads data; market index is each day's average from validation.csv (features only).

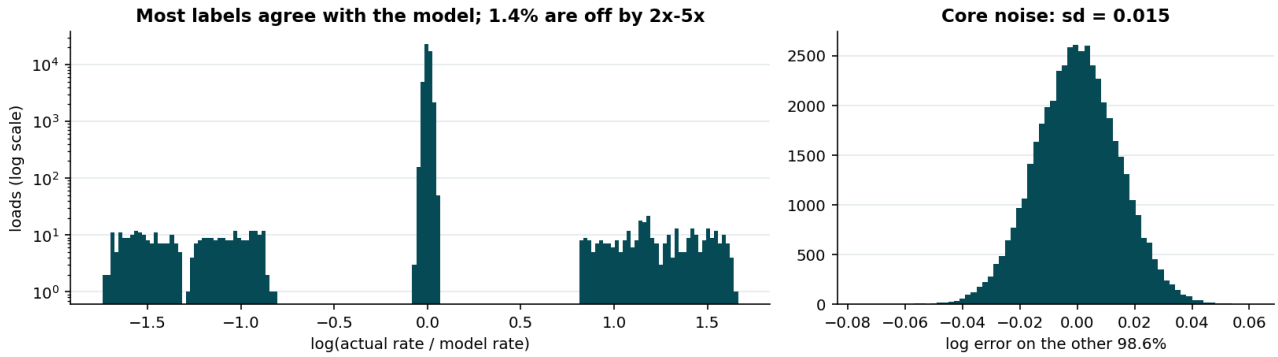


Figure 2. Error of the final model on every training load. The two small humps at about +1.2 and -1.3 are the corrupted labels.

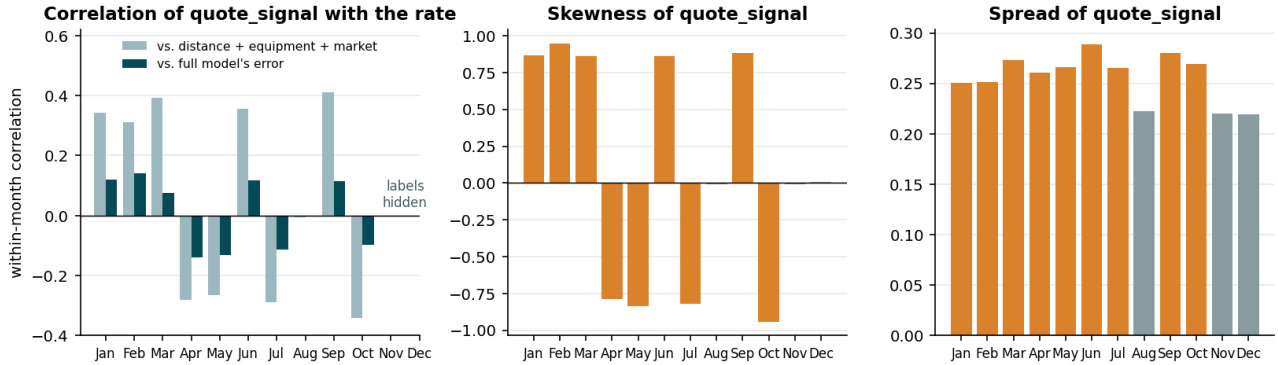


Figure 3. Left: within-month correlation of quote_signal with the rate error (labels are not available for Nov and Dec). Middle and right: the shape of quote_signal itself, which in Nov and Dec matches August, the month with no effect.

4. Train / validation split and how I validated

Why not a random split. The final test is the two months after the last training day and includes new cities. A shuffled split lets the model see the same days and the same lanes on both sides, so it rewards memorisation of month-specific behaviour, which is exactly what fails on the validation months:

Model	Shuffled 5-fold (log RMSE)	Out-of-time (log RMSE)	Shuffled MAPE	Out-of-time MAPE
Boosted trees on all raw columns	0.0240	0.0758	1.82%	5.45%
Final model	0.0163	0.0188	1.30%	1.51%

What I used instead.

- **Expanding-window out-of-time folds.** Six folds. Each trains on every development row before a cut-off and tests on the following two months (cut-offs 1 Apr, 1 May, 1 Jun, 1 Jul, 1 Aug, 1 Sep), the same shape as the real task. Training sizes run from 14,291 to 38,477 loads. Four of the six test windows contain a quarter-end ramp, which checks that the ramp learned from earlier quarters carries forward.
- **Unseen-city test.** Eight random cities (the same count as in the real validation set) are removed from training entirely, and the model is scored on September-October loads that touch them. The same rows are scored by a model that did see those cities as a control. Five random draws.
- **Clean and all-row metrics.** The corrupted rows are flagged once from a reference fit and held fixed, so every candidate is judged on the same rows. I report both: 'clean' shows what the model can control, 'all rows' is what a scorer using raw labels will see.
- **No tuning on the final set.** It has no labels. Design choices were made on the fold average, and the boosting settings were barely sensitive (during development, log RMSE stayed between 0.0196 and 0.0201 across the settings I tried), so there is little to over-fit.

- The submitted model is the same pipeline refit on all 48,000 development loads.

Out-of-time results, mean of 6 folds	Log RMSE	MAPE (clean)	MAE (clean)	MAPE (all rows)
Median rate-per-mile by equipment	0.1154	8.33%	$165.0	10.49%
Boosted trees on all raw columns	0.0758	5.45%	$120.4	7.66%
Final model without geography	0.0193	1.55%	$36.6	3.89%
Final model without lane correction	0.0197	1.58%	$37.3	3.91%
Final model + quote_signal	0.0610	4.29%	$90.1	6.53%
Final model	0.0188	1.51%	$35.7	3.84%

Log RMSE is the typical error as a fraction of the rate (0.019 is about 1.9%). The no-geography row is close to the final model here because the city and lane offsets already cover every city seen in training; coordinates matter for cities that were not seen (below).

Final model by fold	Apr-May	May-Jun	Jun-Jul	Jul-Aug	Aug-Sep	Sep-Oct
Log RMSE	0.0200	0.0176	0.0175	0.0229	0.0169	0.0180
MAPE (clean)	1.60%	1.39%	1.40%	1.90%	1.35%	1.43%
MAE (clean)	$37.9	$33.8	$33.5	$43.9	$31.4	$33.9
MAPE (all rows)	4.14%	3.67%	3.48%	4.21%	3.62%	3.95%

Unseen cities. With the cities removed from training, the final model's log RMSE is 0.0214 (MAPE 1.69%), against 0.0181 for the same rows when the cities were known. Without coordinates it would be 0.0288: the geography features are what make new cities priceable. Only 12% of the validation loads involve a new city.

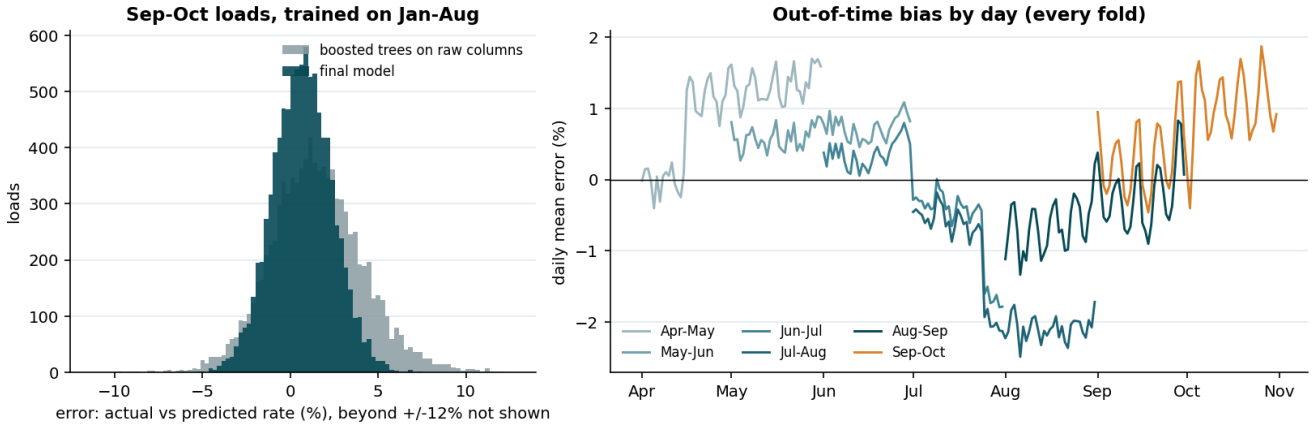


Figure 4. Left: error on September-October loads for models trained on January-August. Right: the daily mean error in every out-of-time fold (the level drifts, but stays within about 2%).

5. Model and the reasoning behind it

Target. log(rate / distance). Rate scales with distance almost proportionally, so the rate per mile is the natural quantity, and in logs the effects are additive, which is what the data shows.

- **G: gradient-boosted trees** (scikit-learn *HistGradientBoostingRegressor*) on distance, equipment, weight and geography (pickup and delivery coordinates, their midpoint and difference). Nothing in G depends on the date, so it cannot memorise a month. This is where the non-linear distance, weight and regional structure is learned.
- **T: a linear calendar-and-market part** with an intercept, a daily drift, a quarter-end ramp per equipment type, log market index and market index x log distance. Trees cannot extrapolate: a tree given the date would predict the last October level for every later day. A linear term continues the drift and applies the Mar/Jun/Sep ramp to December.
- **Back-fitting.** G and T are fitted alternately (four rounds) so that neither absorbs the other's signal; corrupted labels are detected after the first pass and the fit is repeated without them.

- **Shrunk city and lane offsets.** A ridge regression on pickup city, delivery city and directed lane, fitted on what G and T leave behind. It improves every fold, and cities or lanes that were never seen simply receive no offset.
- **Why not a bigger model.** The table above shows the structure, not model capacity, is what matters: the raw-column boosted-tree model has more freedom than mine and is 4 times less accurate out of time (0.076 against 0.019). Tuning the boosting settings moved the result by about 0.0005.

Fitted calendar and market terms (log scale)	Value
Intercept	-0.0465
Drift per 100 days	+0.0199
Quarter-end ramp, Dry Van	+0.0235
Quarter-end ramp, Reefer	+0.0493
Quarter-end ramp, Flatbed	+0.0773
Market index elasticity	+0.1535
Elasticity change per unit of log distance	+0.0153

6. The fixed December chart

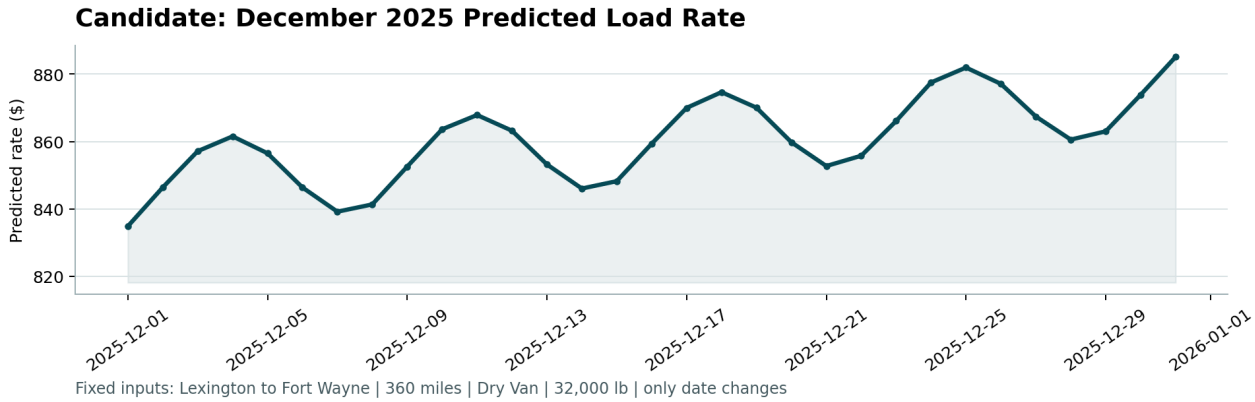


Figure 5. candidate_december.png, produced by the provided score.py from the model's predictions.

Only the date varies (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb). The predicted rate goes from $835 on 1 December to $885 on 31 December, a rise of 6.0%. Three things shape it: the quarter-end ramp for Dry Van (2.3% across the month), the drift, and a weekly cycle of about +/-1.6% that comes from the market index (lowest around Sunday and Monday, highest mid-week). The range is $835 to $885. For context, the 21 Dry Van loads on this lane in the development data (distance 351-379 miles) paid $758 to $934, median $808.

The chart needs two inputs the file does not contain: city coordinates, taken from the loads data, and the market index for each December day, taken as the average of that day's loads in validation.csv. Both are features, not labels. I did not add a Christmas dip or any other holiday shape, because no holiday effect is visible in the January-October data.

7. Assumptions and risks

- **The December ramp rests on three earlier quarter-ends** (March, June, September), which agree closely (pooled over equipment: 3.8%, 3.9% and 4.4%, the same ramp within about half a point). If December behaves differently, the end-of-month predictions are the part that moves.
- **Level drift is the main uncertainty.** The drift is estimated as a straight line. Out of time, the day-by-day error in the overall level was 0.6% to 1.7% (rms) by fold, worst in Jul-Aug, when the drift paused. During development, anchoring the level to the latest weeks or freezing the drift did not do better, so I kept the simple version.

- **quote_signal in Nov-Dec is an inference, not a measurement.** I cannot see the validation labels. The evidence is that its distribution matches August, the one month with no effect, and that using it is costly when the month's regime changes. If labels were available I would test it on the first weeks of November before trusting it either way.
- **The validation labels probably contain the same kind of corrupted rates.** At the training rate of 1.4%, even a near-perfect model scores about 2.3 percentage points worse on raw labels (3.8% against 1.5% here). Squared-error metrics will be hit harder than absolute ones.
- **Not modelled:** any effect of holidays, weather or events outside the January-October window; changes in the lane mix; interactions of the ramp with geography (I checked distance and pickup latitude and found none).

8. Reproducing this work

Code, dependencies and instructions are in the repository README. In short: `pip install -r requirements.txt`, `python scripts/train_and_predict.py`, then `python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv`. The validation tables, figures and this report are regenerated by `scripts/run_validation.py`, `scripts/make_figures.py` and `scripts/build_report.py`.